

Архитектурное описание системы

1. Общие сведения о системе

Разрабатываемая система представляет собой однопользовательскую игру в жанре roguelike с консольным интерфейсом. Игрок управляет персонажем с клавиатуры, перемещается по карте уровня, взаимодействует с мобами и предметами. Предметы могут находиться на карте, подбираться в инвентарь, а также надеваться и сниматься, надетые предметы влияют на характеристики персонажа. Игра состоит из последовательности уровней, уровни могут быть заранее определёнными (загружаться из набора уровней) или генерироваться процедурно. Игра завершается победой при прохождении всех уровней либо поражением при смерти персонажа.

2. Architectural drivers

При проектировании системы учитываются следующие факторы. Архитектура должна быть расширяемой, так как дальнейшие задания предполагают развитие механик и усложнение логики. Требуется высокая скорость разработки, компоненты должны быть слабо связаны, чтобы изменения в генерации уровня, данных или рендере не приводили к переработке всего приложения. Интерфейс должен работать без заметных задержек при вводе и перерисовке. Дополнительным ограничением является простота реализации и понятность структуры для учебного проекта.

3. Роли и случаи использования

В системе присутствует один основной актор - игрок. Игрок запускает приложение и взаимодействует с игрой через клавиатуру. Система отображает состояние уровня и результаты действий пользователя.

3.1. Прецедент П1. Игра в Roguelike игру

Основной исполнитель: игрок.

Заинтересованные лица и их требования: игрок хочет получить удовольствие от игры и проникнуться духом старых игр.

Основной успешный сценарий.

1. Игрок запускает игру.
2. Система генерирует (или загружает) первый уровень.
3. Пользователь проходит уровень, уничтожая всех мобов.
4. Система предлагает пользователю перейти на новый уровень.
5. Пользователь подтверждает переход.

Действия, описанные в пп. 2-5, повторяются, пока пользователь не пройдет все уровни.

6. Система поздравляет пользователя с прохождением игры.

Расширения.

*а. При закрытии пользователем терминала с игрой.

1. Приложение завершается, не делая при этом никаких дополнительных действий.

4. Диаграмма компонентов и описание компонентов

Система разделена на несколько логических подсистем. Подсистема генерации событий (EventProducers) отвечает за получение ввода и формирование объектов событий. Ядро (Core) содержит состояние игры и инициализацию, а слой контроллеров (Controllers) принимает события и запускает обработку.

Для исключения прямых вызовов бизнес-логики из UI введён слой команд (Commands): контроллер на основе входного события выбирает и выполняет команду (ICommand.execute()). Это позволяет централизованно управлять действиями пользователя, а также упрощает расширение (добавление новых действий через новые команды).

Пакеты Models и Mobs содержат доменные сущности (игрок, mobs, предметы, комнаты, уровни) и логику поведения мобов. Пакет Repositories предоставляет доступ к конфигурации и предопределённым картам/данным.

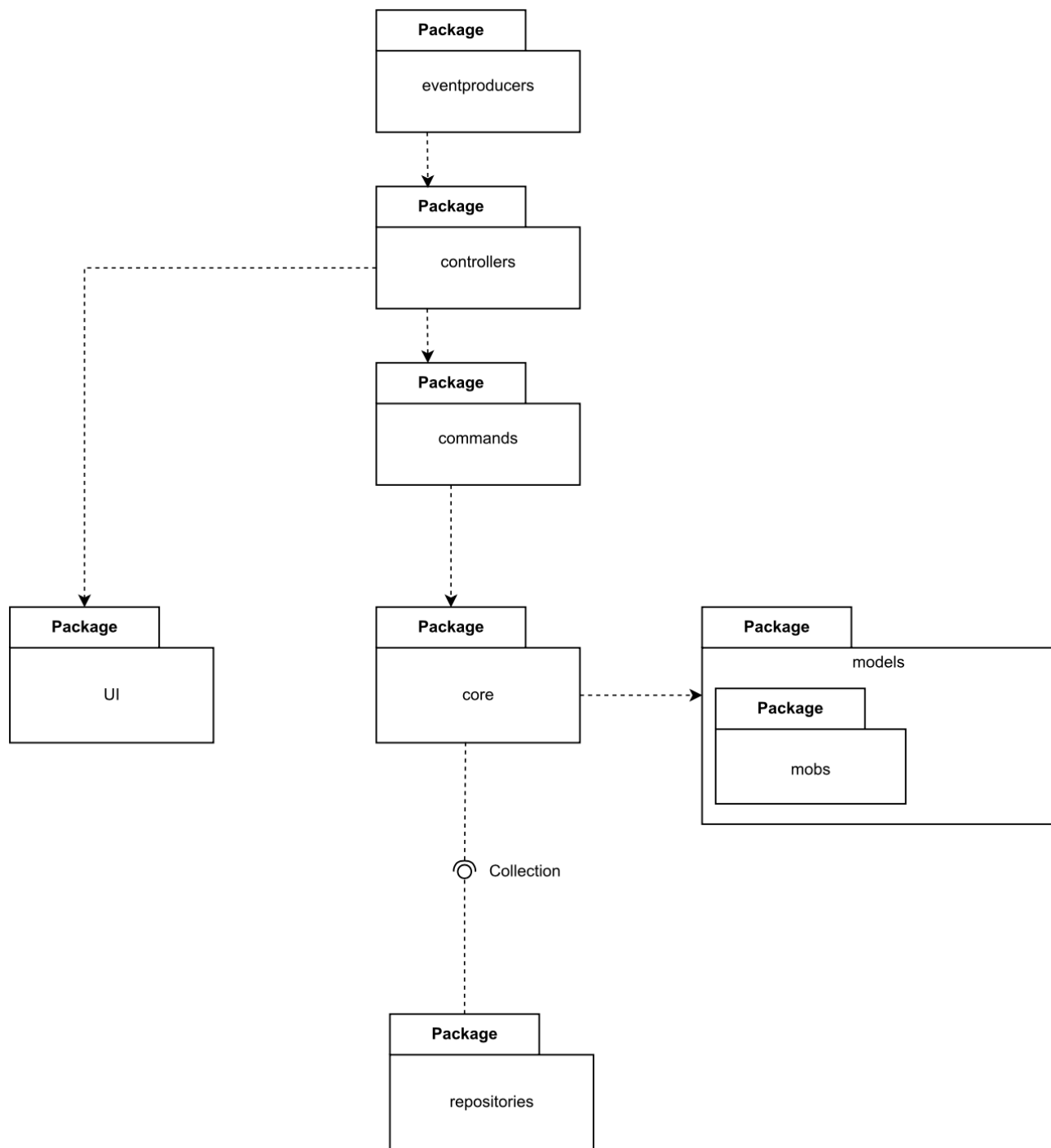


Рис. 1. Диаграмма компонентов (пакетов)

5. Диаграмма классов и логическая структура

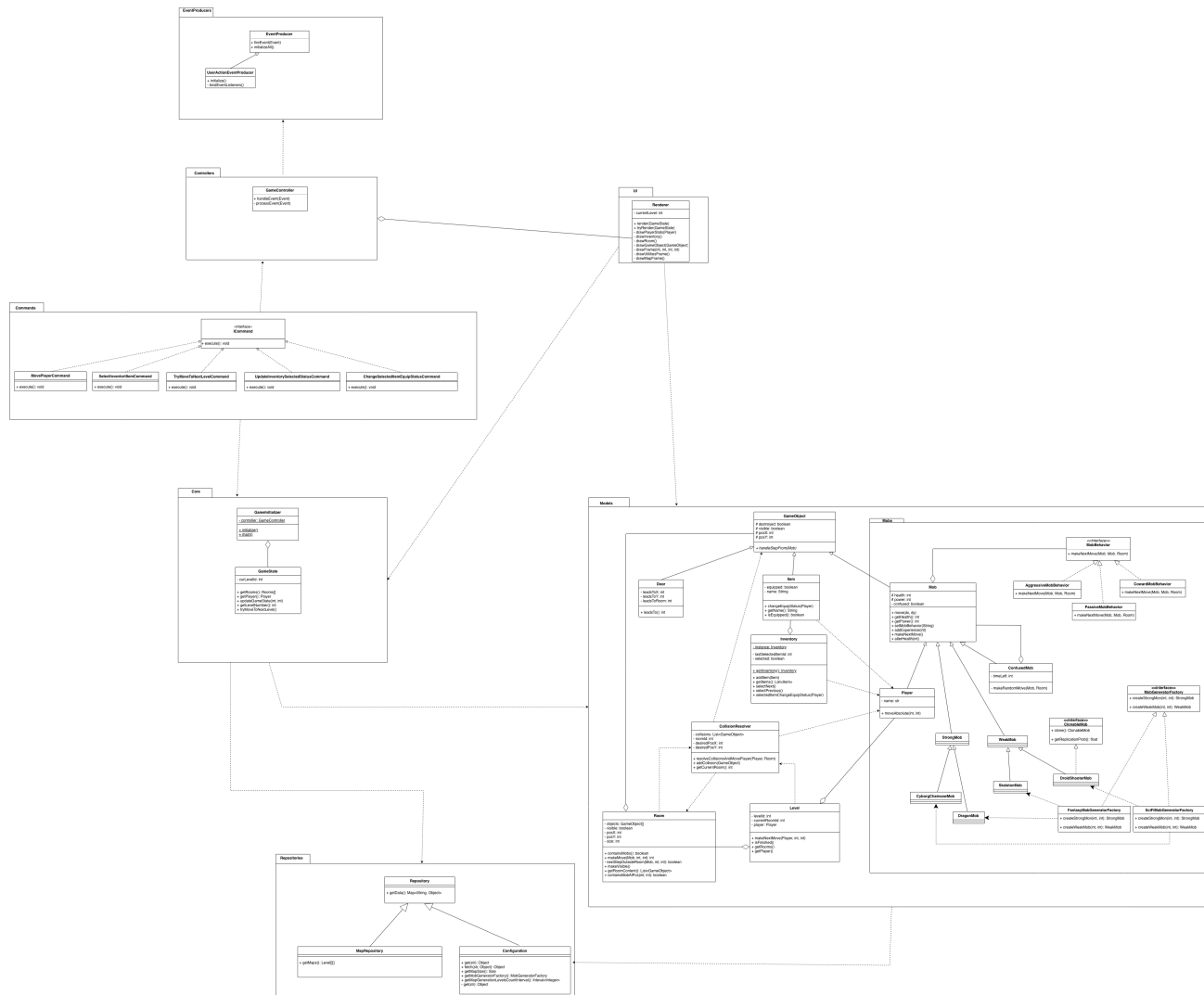


Рис. 2. Диаграмма классов приложения

Генерация событий выполняется объектом `UserActionEventProducer`, который получает ввод пользователя и передает событие в `GameController` (метод `handleEvent(Event)`). Контроллер не выполняет бизнес-логику напрямую, он преобразует событие в соответствующую команду и вызывает `ICommand.execute()`.

Команды инкапсулируют действия пользователя. Например, `MovePlayerCommand` изменяет позицию игрока и инициирует разрешение коллизий на уровне, `TryMoveToNextLevelCommand` пытается переключить

уровень, а команды работы с инвентарём (`SelectInventoryItemCommand`, `UpdateInventorySelectedStatusCommand`, `ChangeSelectedItemEquipStatusCommand`) реализуют выбор предмета и смену статуса «надет/снят».

Состояние игры агрегируется в `GameState` - оно хранит игрока, текущий уровень и текущую комнату, а также предоставляет операции для обновления состояния (например, `updateGameState(int, int)` и `tryMoveToNextLevel()`). Отображение выполняется в `Renderer.render(GameState)`.

Поведение мобов реализуется комбинацией паттернов «Стратегия» и «Состояние». В качестве стратегий используются реализации `MobBehavior` (`AggressiveMobBehavior`, `CowardMobBehavior`, `PassiveMobBehavior`). Поверх стратегий вводится состояние моба, в обычном состоянии моб использует исходную стратегию, а при падении здоровья ниже порога переходит в состояние «паника», где активная стратегия заменяется на трусливую. При восстановлении здоровья выше порога моб возвращается в состояние «всё ок» и восстанавливает исходную стратегию. Переключение состояния вызывается при изменении здоровья (например, внутри `Mob.alterHealth(int)` или эквивалентного метода).

Для сохранения предыдущих механик, временные эффекты (например, оглушение/путаница) поддерживаются декоратором `ConfusedMob`, который изменяет логику движения моба на ограниченное время (`timeLeft`).

Генерация мобов выполняется фабрикой `MobGeneratorFactory` с конкретными реализациями `FantasyMobGeneratorFactory` и `SciFiMobGeneratorFactory`, создающими согласованный набор мобов выбранного «стиля» (например, `DragonMob`, `SkeletonMob`, `CyborgChainsawMob`, `DroidShooterMob`, а также `StrongMob/WeakMob`). Генерация карты реализована через паттерн «Строитель» - `MapGenerator` (или аналогичный строитель)

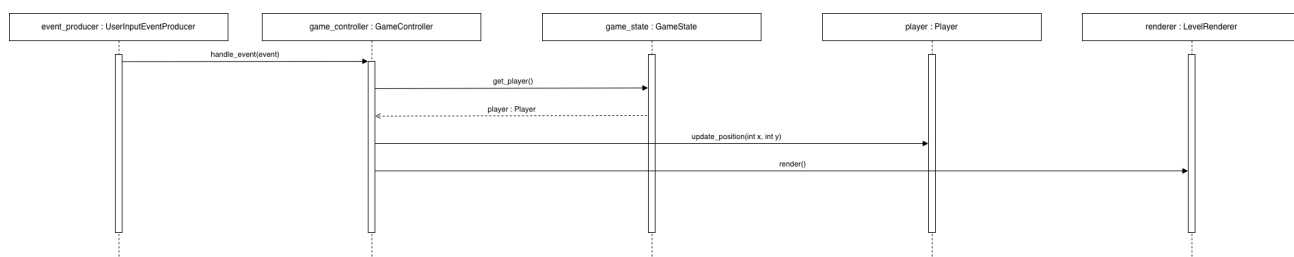
настраивается через Configuration (размер карты, метод генерации, количество уровней и фабрика мобов) и затем формирует итоговую карту.

Для мобов, способных к репликации, используется паттерн «Прототип»: интерфейс ClonableMob определяет метод clone(), а вероятность репликации хранится в параметрах моба (getReplicationProb()).

6. Диаграммы последовательностей (взаимодействия)

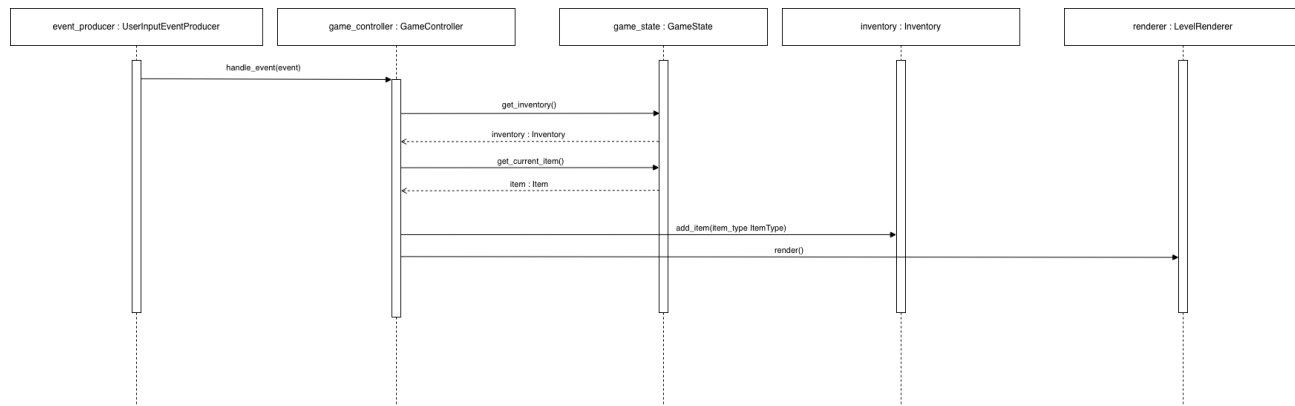
Диаграммы последовательностей фиксируют типовые сценарии работы системы и показывают передачу управления между объектами при обработке событий пользователя.

6.1. Перемещение персонажа



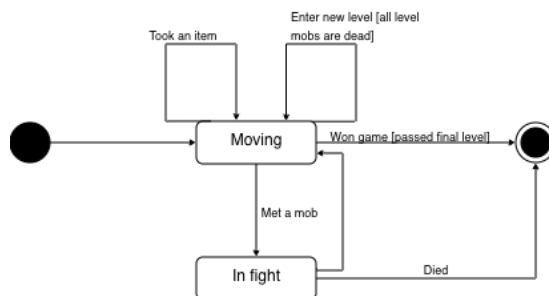
UserActionEventProducer формирует событие и инициирует его обработку в GameController.handle_event(Event). GameController получает игрока через GameState.get_player(): Player и изменяет позицию игрока (например, через метод Player.update_position(int x, int y) или эквивалентный вызов). После изменения состояния GameController вызывает Renderer.render(GameState).

6.2. Подбор предмета



`UserActionEventProducer` формирует событие выбора предмета и передаёт его в `GameController.handle_event(Event)`. `GameController` получает доступ к `Inventory` (через `GameState.inventory` или через аксессор) и добавляет предмет в инвентарь (например, вызовом `Inventory.add_item(...)`). Далее выполняется `Renderer.render(GameState)`.

7. Диаграмма состояний



Состояния игры представлены режимами `Moving` (исследование уровня), `In fight` (бой) и `Game over` (завершение игры). Переход `Moving` > `In fight` происходит при встрече с мобом. После боя возможен переход в `Moving` при победе или переход в `Game over` при смерти игрока. Переход на следующий уровень выполняется при выполнении условия завершения уровня и не меняет основной режим (остается `Moving`), но обновляет текущий `Level`. При прохождении последнего уровня выполняется переход в `Game over` с результатом “победа”.